------------------------- Slip 10 -------------------

Q.1) Write a program that illustrates how to execute two commands concurrently with a pipe.

```c
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>


int main() {

   int fd[2];

   pid_t pid1, pid2;


   // Create a pipe

   if (pipe(fd) == -1) {

      perror("pipe");

      exit(1);

   }


   // First child executes command1 (ls -l)

   pid1 = fork();

   if (pid1 < 0) {

      perror("fork");

      exit(1);

   }


   if (pid1 == 0) {

      // Child 1

      close(fd[0]);              // Close read end

      dup2(fd[1], STDOUT_FILENO); // Redirect stdout to pipe

      close(fd[1]);


      execlp("ls", "ls", "-l", NULL); // Replace with any command

      perror("execlp ls");

      exit(1);

   }
```

```
    // Second child executes command2 (wc -l)
    pid2 = fork();
    if (pid2 < 0) {
        perror("fork");
        exit(1);
    }

    if (pid2 == 0) {
        // Child 2
        close(fd[1]);             // Close write end
        dup2(fd[0], STDIN_FILENO);  // Redirect stdin from pipe
        close(fd[0]);

        execlp("wc", "wc", "-l", NULL); // Replace with any command
        perror("execlp wc");
        exit(1);
    }

    // Parent closes both ends of the pipe
    close(fd[0]);
    close(fd[1]);

    // Wait for both children to finish
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    return 0;
}
```

Q.2) Generate parent process to write unnamed pipe and will write into it. Also generate child process which will read from pipe

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2]; // fd[0] = read end, fd[1] = write end
    pid_t pid;
    char write_msg[] = "Hello from parent!";
    char read_msg[100];

    // Create unnamed pipe
    if (pipe(fd) == -1) {
        perror("pipe");
        exit(1);
    }

    // Fork to create child process
    pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid > 0) {
        // Parent process
        close(fd[0]); // Close unused read end

        // Write message to pipe
        write(fd[1], write_msg, strlen(write_msg) + 1);
        printf("Parent wrote to pipe: %s\n", write_msg);
```

```c
        close(fd[1]); // Close write end after writing

        // Wait for child to finish
        wait(NULL);
    } else {
        // Child process
        close(fd[1]); // Close unused write end

        // Read message from pipe
        read(fd[0], read_msg, sizeof(read_msg));
        printf("Child read from pipe: %s\n", read_msg);

        close(fd[0]); // Close read end
        exit(0);
    }

    return 0;
}
```